



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 8. Дообучение нейросетей (/teach/control/stream/view/id/931478148)

РЕ8.1 Дообучение нейросетей. Эффективное переиспользование модели

Следующий урок (/pl/teach/control/lesson/view?id=338667547)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Уважаемые студенты!

В рамках данного урока могут рассматриваться примеры передачи персональных данных при регистрации на иностранных сервисах, а также обработки и хранения пользовательских данных.

В соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных», обращаем ваше внимание: в демонстрационных примерах и при выполнении домашних

заданий необходимо использовать исключительно тестовые данные или тестовые аккаунты, в которых не указаны персональные данные — используйте вымышленные ники, дополнительные данные и т.п.

Использование иностранных сервисов не является обязательным. Вы вправе выполнять домашние задания на аналогичных разрешённых платформах по вашему выбору.

Результат дня

Сегодня на уроке мы узнаем, как эффективно переиспользовать предобученные модели для решения новых задач с помощью дообучения нейросетей, а также рассмотрим важность качества данных при работе с моделями.

 Время чтения и просмотра: ~1 час

 Время выполнения задания: ~1 час

Теория на сегодня

Дообучение нейросетей



Как эффективно переспользовать
модели для решения новых задач



zerocoder.ru

- Как работает механизм дообучения
- Преимущества Transfer Learning
- На что стоит обращать внимание при дообучении моделей

🔗 Презентация эксперта:

<https://docs.google.com/presentation/d/1oQBfep7gnVANOpU-5wztSBAvuqH4chVVEzwFeSabLXg/edit?usp=sharing>

(<https://docs.google.com/presentation/d/1oQBfep7gnVANOpU-5wztSBAvuqH4chVVEzwFeSabLXg/edit?usp=sharing>)

1. Transfer Learning

Нейросети как “черный ящик”



Получить персональное
видео

КОТ

zerocoder.ru

Сегодняшний урок посвящен дообучению нейросетей. Мы обсудим, как эффективно переиспользовать модели для решения новых задач.

Эксперт — Иван Родин:

- Постдок исследователь, Университет Катании;
- компьютерное зрение, анализ длинных видео, FPV, Action Anticipation;
- исследовательские проекты с Intel, Philips;
- публикации на CVPR, ICASSP, NAACL, CVIU

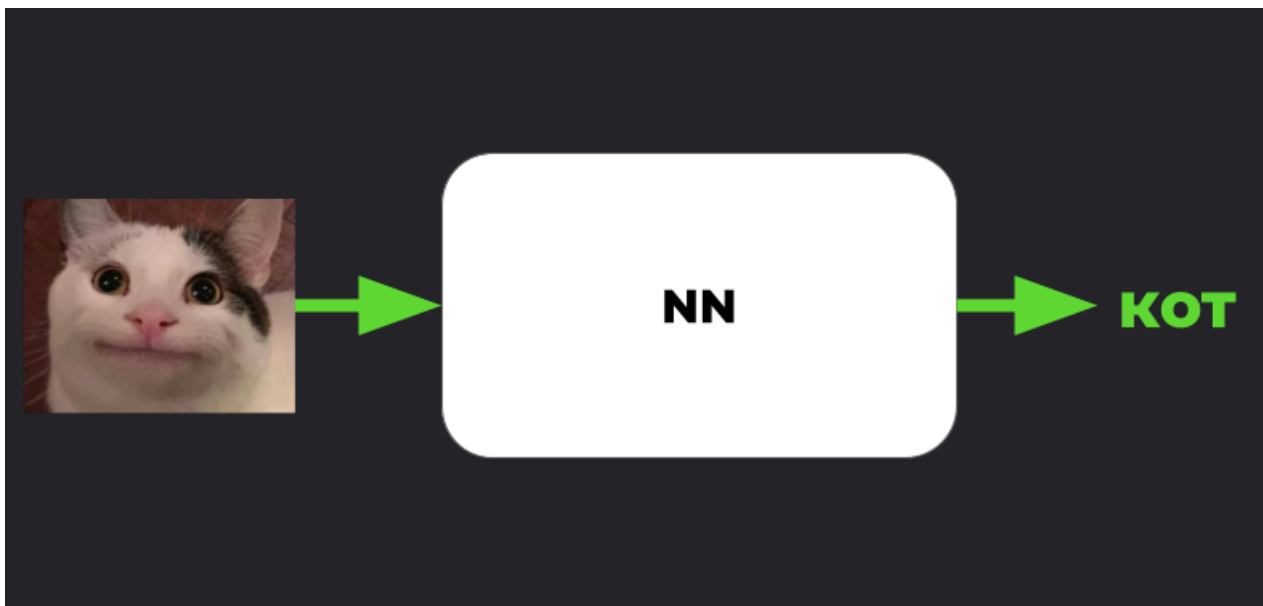
LinkedIn: <https://www.linkedin.com/in/mr-ivan-rodin/>
(<https://www.linkedin.com/in/mr-ivan-rodin/>)

TG-Channel: https://t.me/aimazing_materials
([http://https://t.me/aimazing_materials](https://t.me/aimazing_materials))

План урока

1. Как работает механизм дообучения.
2. Преимущества Transfer Learning (трансферного обучения).
3. На что стоит обращать внимание при дообучении моделей.

Мы часто воспринимаем нейросети как "чёрный ящик", который просто работает между нашим запросом и выдачей. Нейросети принимают на вход данные, такие как изображения, и выдают предсказания, например, определяя, что изображено на картинке.



Нейросети обучаются на ошибках. Они не могут сразу дать правильные предсказания без обучения на большом количестве примеров. Если нейросеть ошибается, например, приняв изображение пса за тигра, эта ошибка приводит к обновлению модели. После обновления весов нейросеть учится, и вероятность повторения ошибки снизится.



Нейросеть можно рассматривать как параметризованную функцию, обозначаемую в литературе как $f(x, w)$, где x — входные данные (например, кодирующий изображение вектор или матрица изображения), а w — веса или параметры модели. Большие языковые модели могут иметь миллиарды параметров.

Модель обрабатывает данные слоями. Входные векторы ' x ' обрабатываются первым слоем модели, создавая скрытое представление данных. Следующие слои продолжают обработку выходов предыдущих слоёв. Чем больше слоёв, тем глубже модель и тем больше она может извлечь из данных. Термин "глубокое обучение" связан с количеством слоёв в модели.

Слои, которые ближе к ' x ', помогают понять данные, извлечь полезные признаки.

Слои, которые ближе к ' y ', нужны для генерации ответа.

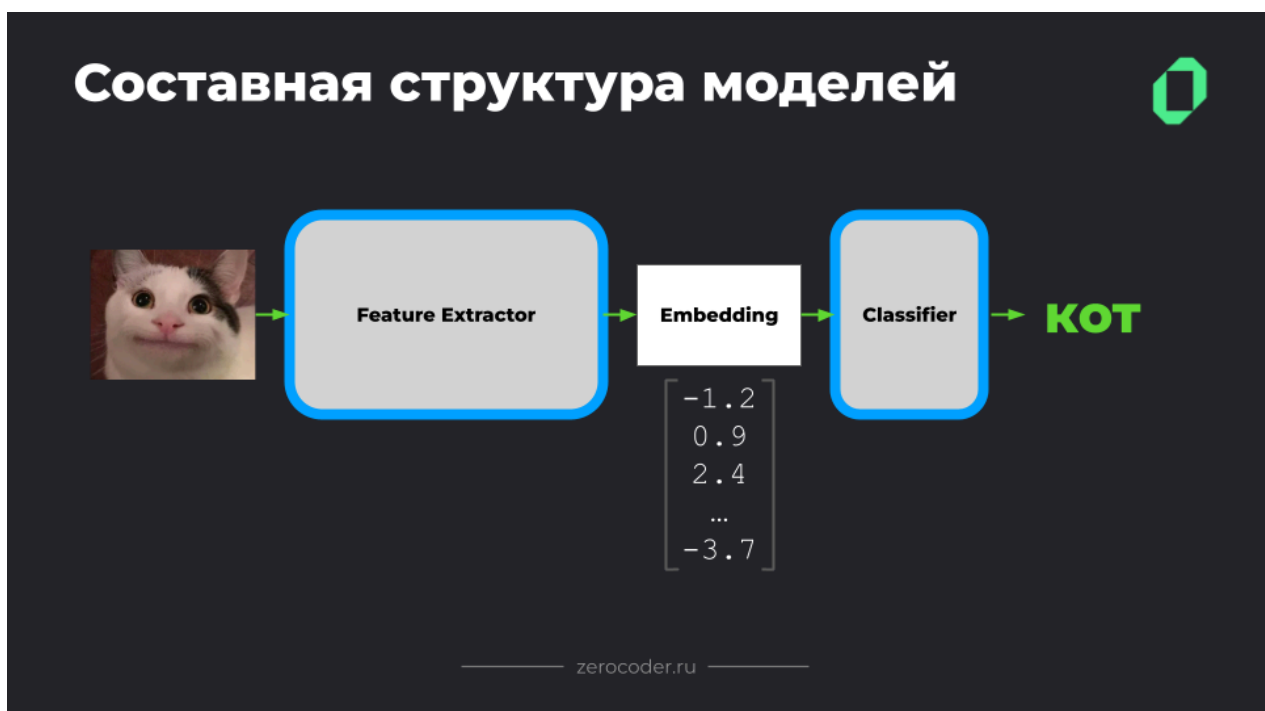


Пайплайн обработки данных с помощью нейросетей можно описать следующим образом:

1. **Фича-экстрактор** обрабатывает входные данные и создает эмбеddинг — вектор, который эффективно кодирует основные характеристики данных.



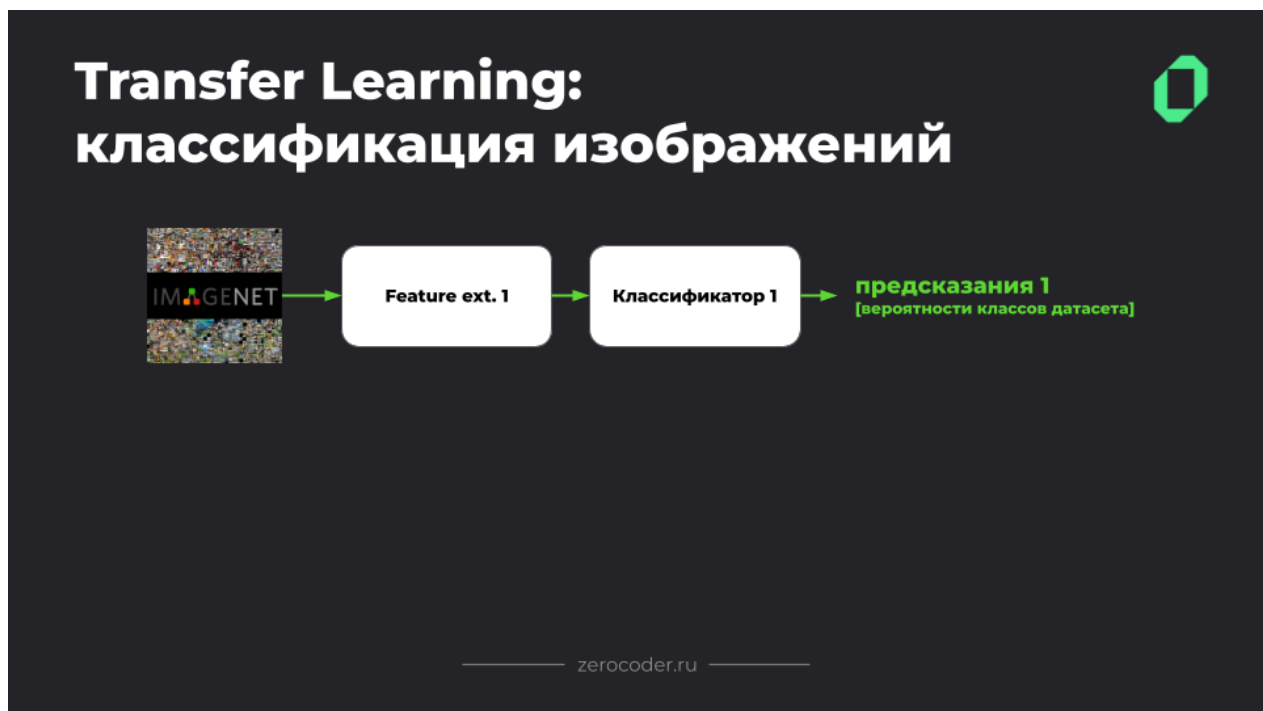
2. **Классификатор** использует эмбеddинг для выполнения финальной задачи, например, классификации изображения.



Фича-экстрактор извлекает признаки из данных. Например, при классификации изображений он может обнаруживать такие признаки, как пушистость, наличие глаз или определенные цвета. Затем эмбединг передается классификатору, который выполняет окончательное предсказание.

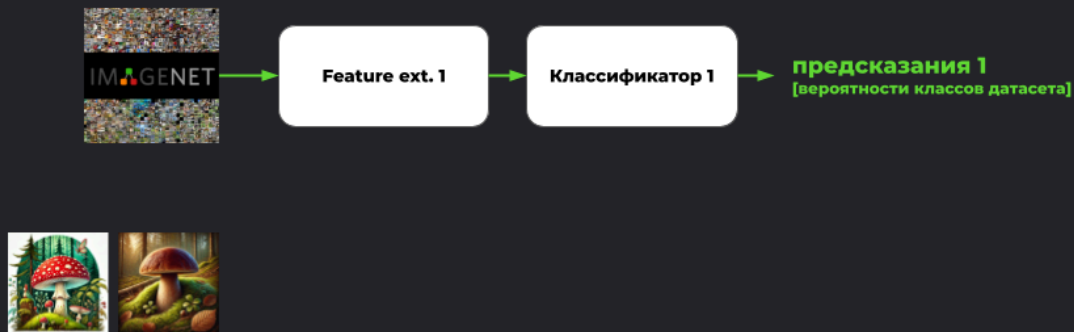
Можно ли переиспользовать фича-экстрактор для новых данных? Ответ — да. Рассмотрим это на примере:

1. **Обучение на ImageNet** — у нас есть датасет ImageNet с миллионами изображений, размеченных по нескольким тысячам классов. Мы обучаем модель, которая состоит из фича-экстрактора и классификатора, на этом датасете.



2. **Новая задача** — нам нужно классифицировать съедобные и ядовитые грибы. У нас есть новый датасет с изображениями грибов и их метками (съедобные или ядовитые).

Transfer Learning: классификация изображений



zerocoder.ru

3. Переиспользование модели:

Transfer Learning: классификация изображений



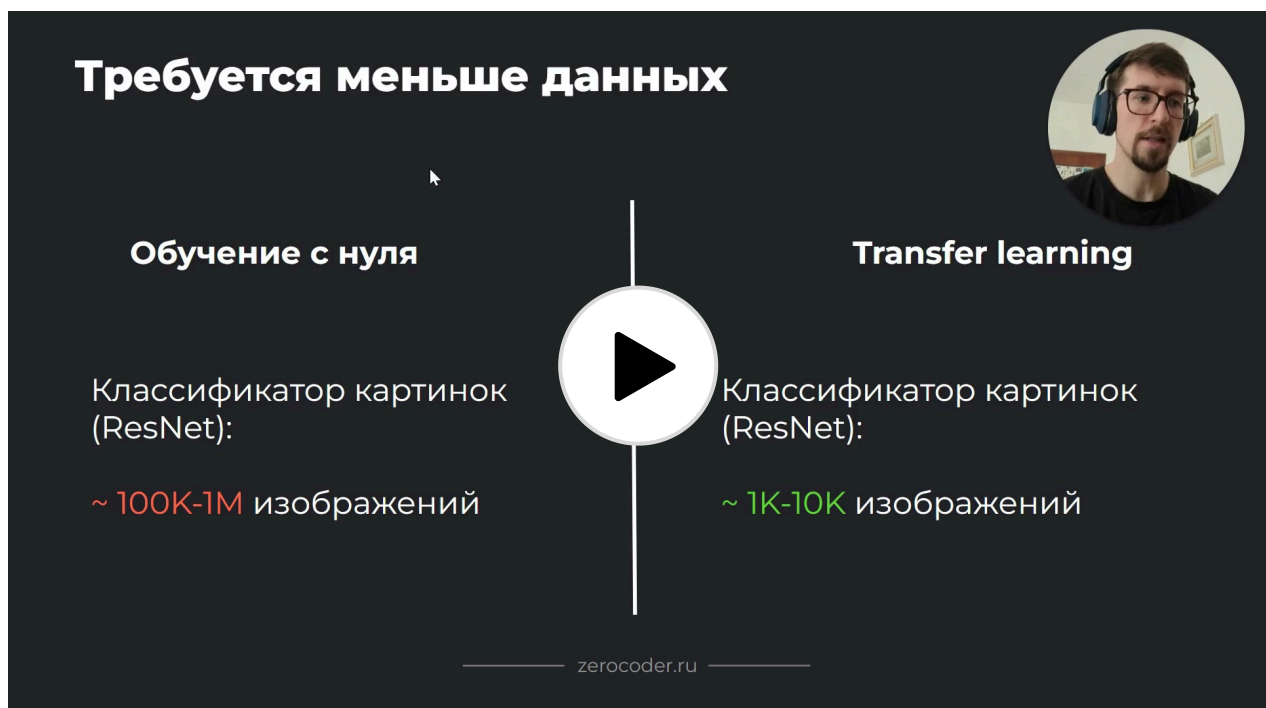
zerocoder.ru

- копируем фича-экстрактор со всеми весами, обученными на ImageNet.
- создаем новый классификатор для новой задачи.
- обучаем новую модель с использованием старого фича-экстрактора и нового классификатора.

Таким образом, мы получаем модель, способную классифицировать съедобные и ядовитые грибы, переиспользуя знания, полученные на предыдущем датасете.



2. Преимущества Transfer Learning



- требуется меньше данных

Одно из ключевых преимуществ трансферного обучения заключается в том, что если мы используем предобученную модель, то нам потребуется значительно меньше данных для её дообучения. Например, если для обучения модели сверточной нейросети ResNet с нуля необходимо несколько сотен тысяч или даже миллионов изображений, то для дообучения предобученной модели на новом датасете достаточно нескольких тысяч или десятков тысяч изображений.

- время на обучение сокращается в несколько раз

Если рассматривать обучение больших языковых моделей, таких как BERT LLM, обучение с нуля может занять около недели. Однако, если использовать уже предобученную модель и дообучить её на новых данных, этот процесс может занять всего 10 часов, иногда чуть больше или меньше, но в любом случае это значительно быстрее.

- снижение требований к ресурсам. Причины:
 - **инициализация весов** — при обучении модели с нуля все веса инициализируются случайным образом, и в процессе обучения обновляются все веса модели. Это требует значительных ресурсов, особенно памяти.
 - **фиксация весов** — в случае трансферного обучения можно "заморозить" веса фича-экстрактора, что означает, что эти веса не будут обновляться в процессе обучения. Когда происходит forward pass, веса используются для предсказания, но при backward pass (обновление весов на основе градиентов) фича-экстрактор не затрагивается. Обновляются только веса классификатора.
 - **сокращение параметров** — классификаторы, последние слои модели, обычно содержат гораздо меньше параметров, чем фича-экстракторы. Это приводит к экономии ресурсов и снижению требований к видеокартам.
- адаптация моделей к новым задачам. Примеры:
 - **классификация изображений** — предположим, у нас есть модель, обученная на классификацию изображений на датасете ImageNet. Мы можем скопировать фича-экстрактор, заморозить его и прикрутить новый классификатор для классификации изображений из нового датасета.
 - **генерация подписей к изображениям** — вместо классификатора мы можем прикрутить большую языковую модель, которая будет генерировать текстовые описания изображений. Например, обучив

модель на парах изображение-текст, можно создать систему для генерации подписей к изображениям.

- **детектирование объектов** — модель можно адаптировать для детектирования и локализации объектов на изображении.
- **задачи регрессии** — можно взять модель, обученную на классификацию, и адаптировать её для задач регрессии. Например, модель, которая изначально классифицировала знаменитостей, можно дообучить для предсказания возраста человека на фотографии.

Одно из важных преимуществ трансферного обучения состоит в том, что этот подход позволяет существенно **снизить риск переобучения модели**. Переобучение (overfitting) происходит, когда модель слишком хорошо запоминает тренировочные данные, но плохо обобщает на новые данные. Представим ситуацию с датасетом изображений ядовитых и съедобных грибов, содержащим пару тысяч изображений. Если мы обучаем нейросеть классифицировать эти изображения с нуля, она может отлично справляться с задачей на этих данных, классифицируя их с точностью 99%. Однако, при встрече с новыми изображениями тех же грибов, но с других ракурсов или при другом освещении, результаты могут значительно ухудшиться.

Это происходит потому, что модель не столько учится, сколько запоминает тренировочные данные. Она буквально "зазубривает" датасет, и при малейшем изменении в исходных данных модель сразу "сыпется". Это можно сравнить со студентом, который вызубрил билеты, но не готов к неожиданным вопросам на экзамене.

Трансферное обучение помогает решить эту проблему. Когда мы используем предобученную модель, например, на большом датасете ImageNet, модель уже видела множество различных изображений и научилась выделять значимые признаки, а не просто запоминать данные. Это снижает риск переобучения и делает модель более устойчивой к новым данным, что особенно важно для реальных приложений.

Трансферное обучение применяется практически везде. Почти любая модель, с которой мы сталкиваемся в повседневной жизни, скорее всего, была предобучена на одном датасете и затем нафайн-тюнена для конкретной задачи. Например:

- **автопилоты** — модели, используемые в автопилотах, предобучены на детектирование объектов и затем дообучены на специфические задачи, например, на распознавание автомобилей и пешеходов в определённом городе;
- **большие языковые модели** — эти модели, такие как GPT или BERT, изначально обучаются на больших корпусах текстов из интернета, учатся предсказывать следующий токен в последовательности. Затем они дообучаются на более специфичные задачи, такие как суммаризация текстов, перевод, генерация кода. После этого мы получаем мощные инструменты, которые можно использовать в различных приложениях.

Если мы хотим использовать предобученную модель, нам не нужно обучать её с нуля. Предобученные модели можно найти:

- в документации библиотек PyTorch или TensorFlow;
- на ресурсах вроде PapersWithCode.com (<http://PapersWithCode.com>), где есть научные статьи с описанием моделей и ссылками на их веса;
- Hugging Face, где легко найти и использовать предобученные модели.

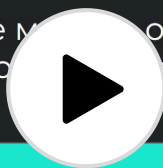
3. Как происходит дообучение модели

Train / Validation / Test



!ВАЖНО!

Всегда, когда (до-)обучаете модель, оценивайте ее качество не на тех примерах, на которых обучаете!



Train (~80%)

Validation

Test

zerocoder.ru

При дообучении модели важно оценивать её качество не на тех примерах, на которых она обучалась. Это делается для того, чтобы избежать переобучения. Модели могут запоминать данные, но это не значит, что они будут хорошо работать на новых данных.

Разделяем датасет: 80% используем для обучения и оставшиеся 20% — для валидации модели. Это основное правило машинного обучения и глубокого обучения. Такой подход помогает убедиться, что модель будет хорошо работать не только на обучающих данных, но и на новых, ранее не виденных данных.

Train (~80%)

Validation

Test

При обучении модели с нуля и при дообучении важно проверять данные, с которыми идёт работа. Независимо от того, насколько мощной или хорошо предобученной является модель, качество её работы будет напрямую зависеть от качества входных данных. Существует выражение "garbage in, garbage out", что означает: если на вход модели подаются некачественные данные, результат тоже будет некачественным.

Пример из личного опыта:

Один из студентов работал над дипломным проектом по классификации изображений с использованием глубокого обучения. Он нашёл датасет на Kaggle, содержащий изображения различных типов кожи (нормальная, жирная и сухая). Его цель состояла в обучении модели для определения типа кожи. Он использовал хорошую предобученную модель, но при обучении на этом датасете получал точность классификации около 80% на валидационной выборке.

Этот результат был странным, учитывая, что задача не должна быть сложной для нейросети. Проблема оказалась в том, что студент не изучил датасет вручную. При более внимательном рассмотрении оказалось, что многие изображения были плохо размечены. Некоторые изображения из класса "сухая кожа" были маркетинговыми анкетами или рекламными фотографиями, не отражающими действительное состояние кожи.

Это яркий пример того, как некачественные данные могут испортить результат. Нужно тратить время на проверку данных. При работе с текстами нужно прочесть несколько примеров и проверить разметку. Если с изображениями — просмотреть их, чтобы убедиться в корректности распределения классов.

Требования к данным:

- входные данные могут быть из нового распределения, но должны быть такой же модальности.

Важно, чтобы входные данные соответствовали формату, на котором была предобучена модель. Например, модель, обученная на изображениях, не сможет правильно обрабатывать аудиодорожки. Если же мы хотим адаптировать модель для другой задачи, например, от классификации изображений к определению возраста по фотографии, нам нужно изменить выходные слои модели.



- если меняется тип (или размерность) выходных данных, нужно адаптировать последние слои (“голову” модели).

Если меняется количество классов или тип задачи, необходимо вручную адаптировать модель, изменить количество выходных нейронов и, возможно, структуру слоев, отвечающих за предсказания.

Исходная задача	Новая задача	Действие
Классификация: 1000 классов, Imagenet	Классификация: 1000 классов	Оставляем “голову” без изменений
Классификация: 1000 классов, Imagenet	Классификация: 3 класса	Меняем количество выходов модели
Классификация: 1000 классов, Imagenet	Регрессия (возраст по фото)	Меняем “голову” основательно

- данные должны удовлетворять ограничениям модели.

Нужно учитывать физические ограничения модели. Например, модели для классификации изображений могут иметь ограничения на минимальный и максимальный размер изображения. Большие языковые модели имеют ограничения на максимальную длину

текста. Всегда нужно читать документацию к моделям и обращать внимание на сообщения об ошибках, которые могут указывать на такие ограничения.

Пример:

- MAX_INPUT_LENGTH
 - MAX_IMAGE_SIZE
 - MIN_IMAGE_SIZE
- иногда нужно выполнять нормализацию данных, чтобы модель адекватно обрабатывала сигналы.

Для корректной работы модели часто требуется нормализация данных. Например, изображения могут нуждаться в нормализации по каналам яркости. Если ведётся работа с медицинскими изображениями, где диапазон яркости может быть значительно больше, чем у обычных фотографий, нужно адаптировать модель для работы с такими данными.

Пример:

- Нормализация изображений по каналам;
- Нормализация аудиосигналов.

Существует несколько подходов к дообучению моделей:

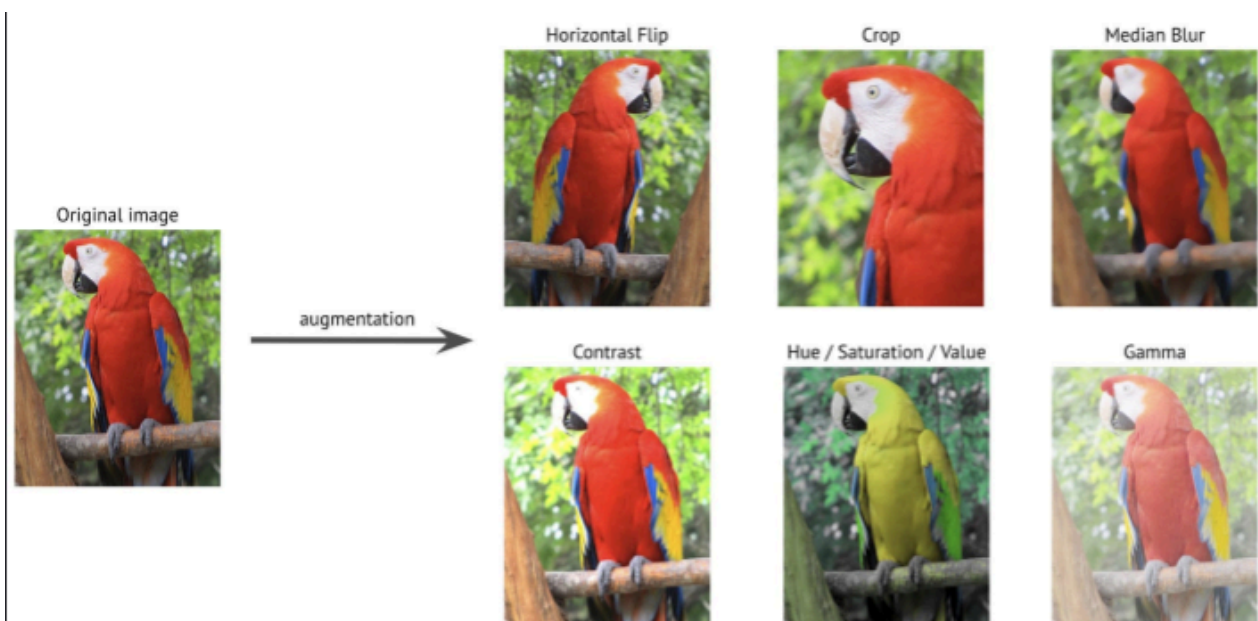
1. **Инициализация** — копирование предобученной модели, добавление нового классификатора и обучение всей модели целиком на новых данных. Это позволяет сохранить предобученные веса и использовать их для более быстрого обучения.
2. **Заморозка весов** — копирование предобученных весов и замораживание их, то есть предотвращение их изменения в процессе обучения. Это уменьшает требования к вычислительным ресурсам и ускоряет обучение, но может снизить гибкость модели.
3. **Частичная заморозка** — копирование предобученных весов и их изменение с небольшим learning rate. Это позволяет модели адаптироваться к новым данным, сохраняя большую часть знаний

из предобученного состояния.

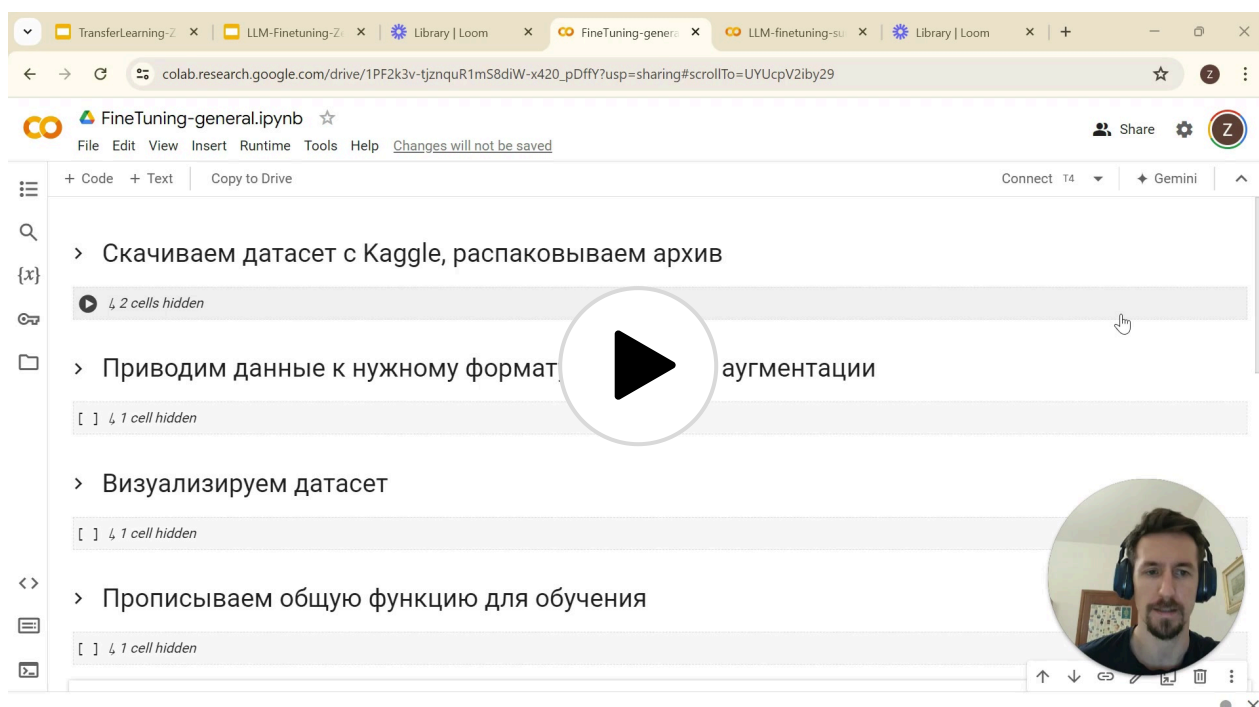
learning rate (lr) — это гипер-параметр, отвечающий за то как сильно меняются веса в процессе обучения.

Для улучшения качества дообучения можно использовать различные техники, такие как аугментация данных.

Аугментация данных включает в себя различные преобразования изображений, такие как зеркальное отображение, увеличение, вращение, изменение яркости и контрастности, добавление шума и другие методы. Это помогает создать более разнообразные данные, что делает модель более устойчивой и уменьшает риск переобучения.



4. Практика: процесс создания и дообучения модели классификации изображений



⚙️ Проект эксперта:

<https://colab.research.google.com/drive/1ZuOO1hsy72Z6wBpgZjOdFMnKCPv6r7lF?usp=sharing>

(<https://colab.research.google.com/drive/1ZuOO1hsy72Z6wBpgZjOdFMnKCPv6r7lF?usp=sharing>)

В этом практическом занятии мы рассмотрим процесс создания и дообучения модели классификации изображений. Мы будем использовать Google Colab для выполнения всех вычислений и работать с датасетом изображений, представляющих жесты для игры в "Камень, ножницы, бумага".

Для начала нам необходимо настроить среду выполнения в Google Colab, чтобы использовать графические ускорители:

1. Открываем Google Colab и заходим в меню "Среда выполнения" (Runtime).
2. Выбираем "Изменить среду выполнения" (Change runtime type).
3. В открывшемся окне выбираем графический ускоритель (GPU). Обычно дефолтный вариант - T4.
4. Нажимаем **Save**, чтобы применить настройки.

Теперь среда готова для выполнения вычислений на GPU, изменения будут отправляться на видеокарту.

Мы будем использовать датасет "Rock, Paper, Scissors", содержащий фотографии рук, показывающих жесты для игры.

☞ Датасет "Rock, Paper, Scissors":

<https://www.kaggle.com/datasets/drgfreeman/rockpaperscissors>
(<https://www.kaggle.com/datasets/drgfreeman/rockpaperscissors>)

1. Скачиваем датасет, распаковываем архив.

```
# Install Kaggle package
!pip install kaggle >> None

# Upload Kaggle API token (you need to have a kaggle.json file with your
from google.colab import files
files.upload()

# Create a Kaggle folder and move the kaggle.json file there
!mkdir -p ~/.kaggle
!mv kaggle.json ~/.kaggle/

# Change the permissions of the file
!chmod 600 ~/.kaggle/kaggle.json

# Download the Rock Paper Scissors dataset from Kaggle
!kaggle datasets download -d sanikamal/rock-paper-scissors-dataset

# Unzip the dataset
!unzip rock-paper-scissors-dataset.zip
```

2. Приводим данные к нужному формату и добавляем аугментацию. После скачивания датасета и сохранения данных по папкам (ножницы, бумага, камень), нам нужно выполнить несколько преобразований для подготовки данных к обучению модели.

Преобразования указываем в переменной `data_transforms`.
Преобразования для тренировочной и для тестовой выборок отличаются. Тестовой выборке аугментации нужны не так сильно.

Также мы создаём подгрузчики данных, которые будут брать датасеты, применять преобразования и скармливать преобразованные данные нейросети.

```

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
import time
import os
import copy

from tqdm import tqdm

from torch.optim import lr_scheduler

# Define transformations
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224), # приведение к одному размеру
        transforms.RandomHorizontalFlip(), # аугментация - отзеркаливание
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
    ]),
    'test': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
    ]),
}

data_dir = 'Rock-Paper-Scissors'
image_datasets = {x: datasets.ImageFolder(os.path.join(data_dir, x), data_transforms[x]) for x in ['train', 'test']}
dataloaders = {x: torch.utils.data.DataLoader(image_datasets[x], batch_size=32) for x in ['train', 'test']}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'test']}
class_names = image_datasets['train'].classes

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print('Running model on ', device)

```

3. Визуализируем датасет, чтобы убедиться в корректности преобразований и аугментаций:

```

import matplotlib.pyplot as plt
import numpy as np
import torchvision

# Function to show images
def imshow(inp, title=None):
    """Imshow for Tensor."""
    inp = inp.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    inp = std * inp + mean
    inp = np.clip(inp, 0, 1)
    plt.imshow(inp)
    plt.xticks([]) # Remove x-ticks
    plt.yticks([]) # Remove y-ticks
    plt.pause(0.001) # pause a bit so that plots are updated

# Get a batch of training data
inputs, classes = next(iter(dataloaders['train']))

# Make a grid from batch
out = torchvision.utils.make_grid(inputs)

# Plot the images
imshow(out, title=[class_names[x] for x in classes])

```

4. Прописываем общую функцию для обучения. Здесь указываем, как скармливаем данные модель, как обновляем модель, какую используем функцию потерь, какие используем метрики. Это универсальный код для большинства моделей — его можно копировать и адаптировать.

```

# Function to train the model
def train_model(model, criterion, optimizer, scheduler, num_epochs=5):
    since = time.time()
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    for epoch in range(num_epochs):
        print(f'Epoch {epoch}/{num_epochs - 1}')
        print('-' * 10)

        for phase in ['train', 'test']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_corrects = 0

            for inputs, labels in tqdm(dataloaders[phase]):
                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

            if phase == 'train':
                scheduler.step()

        epoch_loss = running_loss / dataset_sizes[phase]
        epoch_acc = running_corrects.double() / dataset_sizes[phase]

```



```

print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

if phase == 'test' and epoch_acc > best_acc:
    best_acc = epoch_acc
    best_model_wts = copy.deepcopy(model.state_dict())

time_elapsed = time.time() - since
print(f'Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s')
print(f'Best val Acc: {best_acc:.4f}')

model.load_state_dict(best_model_wts)
return model

```

5. Создаём две модели: одну будем обучать с нуля, а другую дообучим на предобученной модели. Чтобы использовать модель с публично доступной конфигурацией, используем стандартный модель библиотеки PyTorch. Веса не копируем. Адаптируем количество выходов модели (предусмотрена тысяча выходов).

```

# Load ResNet-18 model trained from scratch
model_ft_scratch = models.resnet18(pretrained=False) # НЕ предобученная м
num_fttrs = model_ft_scratch.fc.in_features
model_ft_scratch.fc = nn.Linear(num_fttrs, len(class_names)) # задаем 3 кл
model_ft_scratch = model_ft_scratch.to(device)

criterion = nn.CrossEntropyLoss() # Задаем функцию потерь

optimizer_ft_scratch = optim.SGD(model_ft_scratch.parameters(), lr=0.001,
exp_lr_scheduler_scratch = lr_scheduler.StepLR(optimizer_ft_scratch, step

# Запускаем обучение на 5 эпох
print("Training ResNet-18 from scratch")
model_ft_scratch = train_model(model_ft_scratch, criterion, optimizer_ft_

# Задаем предобученную на ImageNet модель.
model_ft_pretrained = models.resnet18(weights='IMAGENET1K_V1')

# Замораживаем все слои модели
for param in model_ft_pretrained.parameters():
    param.requires_grad = False

num_fttrs = model_ft_pretrained.fc.in_features
model_ft_pretrained.fc = nn.Linear(num_fttrs, len(class_names)) # При пере
model_ft_pretrained = model_ft_pretrained.to(device) # Отправляем на GPU

# Указываем оптимизатору, что только последний слой надо обновлять
optimizer_ft_pretrained = optim.SGD(model_ft_pretrained.fc.parameters(),

exp_lr_scheduler_pretrained = lr_scheduler.StepLR(optimizer_ft_pretrained

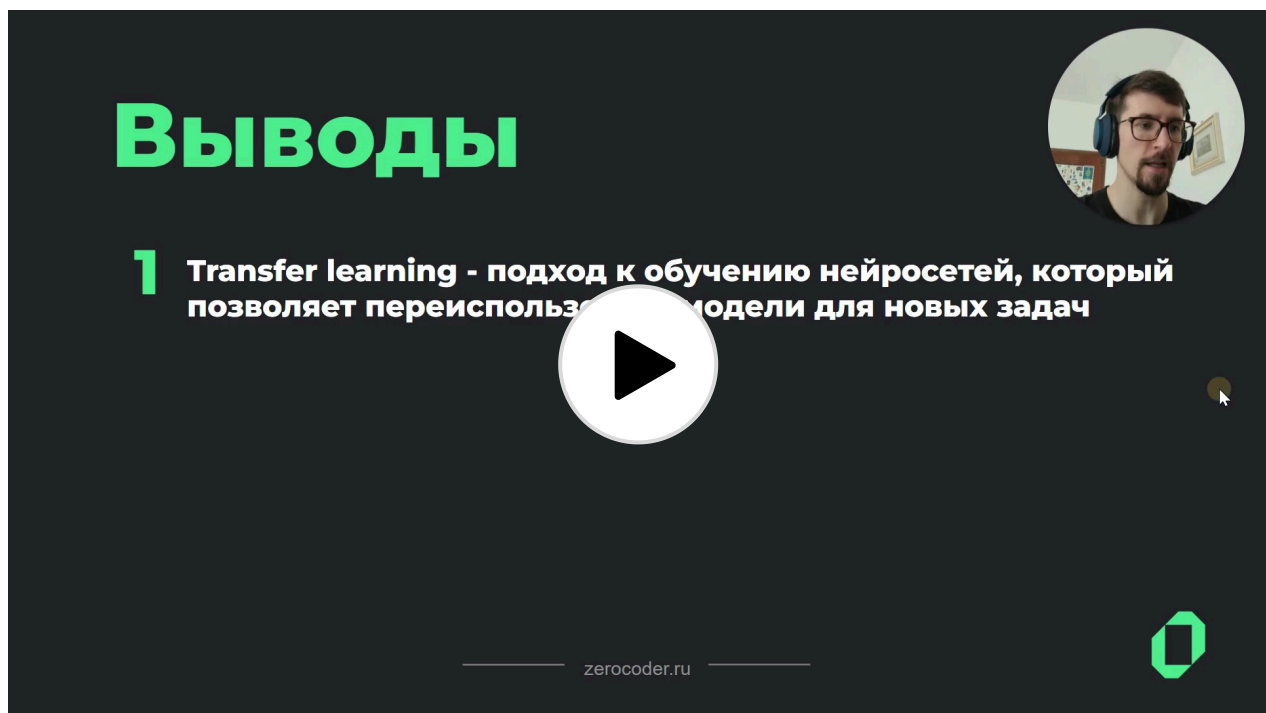
# Запускаем обучение на 5 эпох
print("Fine-tuning pretrained ResNet-18")
model_ft_pretrained = train_model(model_ft_pretrained, criterion, optimiz

```

В итоге мы увидим, что обучение происходит быстро, и промежуточные результаты нам выдаются.

Внизу мы увидим выводы о скорости обучения и точности обеих моделей. Предобученная модель показывает значительное превосходство в точности (90% против 65%). Во времени не такая большая разница, потому что выбранная модель имеет всего 18 слоёв.

Основные выводы урока



- Transfer learning — это подход к обучению нейросетей, который позволяет переиспользовать модели для новых задач.
- Плюсы: требуется меньше данных, ресурсов, времени. Можно достичь лучшего качества.
- При дообучении необходимо учитывать изначальные настройки модели, а также свойства нового датасета. Иногда нужно добавлять некоторые настройки.

Домашнее задание

- Выбери свой собственный датасет, скачай его и приведи данные к нужному формату. Если свой датасет скачать не удастся, то возьми за основу датасет эксперта.

- Пропиши общую функцию для обучения и визуализации датасета.
- Создай модель для обучения и дообучения и сравни их работу.
- В качестве домашнего задания пришли ссылку на свой проект и выводы по работе моделей.

На следующем занятии

На следующем занятии мы рассмотрим тему LLM Fine-tuning.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

PE8.1 Дообучение нейросетей. Эффективное переиспользование модели

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

- Выбери свой собственный датасет, скачай его и приведи данные к нужному формату. Если свой датасет скачать не удастся, то возьми за основу датасет эксперта.
- Пропиши общую функцию для обучения и визуализации датасета.
- Создай модель для обучения и дообучения и сравни их работу.
- В качестве домашнего задания пришли ссылку на свой проект и выводы по работе моделей.



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Следующий урок